low-level threading aren’t good ideas. RPC pretends that a call over the network is no different from a local method call. Every RPC call, when synchronous, needs to block the current thread and wait for a response from the network for the local method call abstraction to work, which can be costly. This approach impedes the goal of using resources efficiently. When RPC is asynchronous, you are forced to pass a callback, which adds complexity. You may end up in the infamous *callback hell*: a callback that generates a callback that generates another, and so on, making the logic of the code hard to follow.

Another problem with this approach is knowing exactly where to scale up or scale out. Multithreaded programming and RPC-based network programming are like apples and pears: they run in different contexts, use different semantics, and run on different levels of abstraction. You end up hardcoding which parts of your application use threads for scaling up and which parts use RPC for scaling out.

Complexity increases significantly when you hardcode methods that work on different levels of abstraction. Quick—what’s simpler, coding with two entangled programming constructs (RPC and threads) or using just one programming construct? This multipronged approach to scaling applications is more complicated than necessary to adapt flexibly to changes in demand.

Spinning up thousands of servers is simple today, but as you’ll see in this chapter, the same can’t be said for programming them. Akka gives you a single abstraction to deal with concurrency and scalability: the actor model, which provides coherent semantics that let you work on your business logic without worrying about whether your program needs to run on a thousand servers or just one.